Q15: Write a final processing pipeline that:

Filters out duplicates.

Fills null prices with 0.

Groups by store_id to calculate total revenue.

```
from pyspark.sql.functions import sum

data = [
    (101, 100.0),
    (101, None),
    (101, 100.0),   # Duplicate
    (102, 200.0),
    (102, None)
]

df = spark.createDataFrame(
    data,
    ["store_id", "price"]
)

result = (
    df
    .dropDuplicates()
    .na.fill(0, subset=["price"])
    .groupBy("store_id")
    .agg(sum("price").alias("total_revenue"))
)

result.show()
```

```
+--------+-------------+
|store_id|total_revenue|
+--------+-------------+
|     101|        100.0|
|     102|        200.0|
+--------+-------------+
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Using inferSchema=true can be risky when a dataset contains inconsistent date formats because Spark may incorrectly identify the column's data type. Some values may be converted to null, treated as strings, or cause parsing errors. This can lead to incorrect analysis and data quality issues.

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Using inferSchema=true can be risky when a dataset contains inconsistent date formats because Spark may incorrectly identify the column's data type. Some values may be converted to null, treated as strings, or cause parsing errors. This can lead to incorrect analysis and data quality issues.

Q15: Write a final processing pipeline that:

Filters out duplicates.

Fills null prices with 0.

Groups by store_id to calculate total revenue.

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

[14]
✓ 0s

```
from pyspark.sql.functions import min, max, avg

data = [
    (100,),
    (200,),
    (300,),
    (400,)
]

df = spark.createDataFrame(
    data,
    ["price"]
)

df.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    avg("price").alias("avg_price")
).show()
```

```
+---------+---------+---------+
|min_price|max_price|avg_price|
+---------+---------+---------+
|      100|      400|    250.0|
+---------+---------+---------+
```

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
data = [
    ("Rahul", "rahul@gmail.com"),
    ("", "aman@gmail.com"),
    ("Priya", None),
    ("Neha", "neha@gmail.com")
]

df = spark.createDataFrame(
    data,
    ["username", "email"]
)

df.filter(
    (df.email.isNotNull()) &
    (df.username != "")
).show()
```

```
+--------+--------------+
|username|         email|
+--------+--------------+
|   Rahul|rahul@gmail.com|
|    Neha| neha@gmail.com|
+--------+--------------+
```

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

```
from pyspark.sql.functions import count

data = [
    ("IT", "Rahul"),
    ("HR", "Priya"),
    ("IT", "Aman"),
    ("HR", "Neha"),
    ("Finance", "Rohit")
]

df = spark.createDataFrame(
    data,
    ["department", "employee"]
)

df.groupBy("department") \
  .agg(count("*").alias("employee_count")) \
  .show()
```

```
+----------+--------------+
|department|employee_count|
+----------+--------------+
|        HR|             2|
|        IT|             2|
|   Finance|             1|
+----------+--------------+
```

Start coding or generate with AI.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
from pyspark.sql.types import TimestampType

data = [
    ("2025-06-20 10:30:00",),
    ("2025-06-21 15:45:00",)
]

df = spark.createDataFrame(
    data,
    ["raw_timestamp"]
)

df = df.withColumn(
    "raw_timestamp",
    df["raw_timestamp"].cast(TimestampType())
).withColumnRenamed(
    "raw_timestamp",
    "event_time"
)

df.show(truncate=False)
```

```
+-------------------+
|event_time         |
+-------------------+
|2025-06-20 10:30:00|
|2025-06-21 15:45:00|
+-------------------+
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

```
data = [
    ("Laptop", 50000),
    ("Mouse", None),
    ("Keyboard", 2000)
]

df = spark.createDataFrame(
    data,
    ["product", "price"]
)

# Fill null prices with 0
df_clean = df.na.fill(0, subset=["price"])

# Calculate average price
df_clean.agg(avg("price").alias("avg_price")).show()
```

```
+-----------------+
|        avg_price|
+-----------------+
|17333.333333333332|
+-----------------+
```

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
data = [
    ("Rahul", 20, "Premium"),
    ("Aman", 35, "Premium"),
    ("Priya", 25, "Basic"),
    ("Neha", 28, "Premium")
]

df = spark.createDataFrame(
    data,
    ["name", "age", "subscription"]
)

df.filter(
    (df.age >= 18) &
    (df.age <= 30) &
    (df.subscription == "Premium")
).show()
```

```
+-----+---+------------+
| name|age|subscription|
+-----+---+------------+
|Rahul| 20|     Premium|
| Neha| 28|     Premium|
+-----+---+------------+
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

```
# Original DataFrame
df.show()

# Drop a column
new_df = df.drop("amount")

# Rename a column
new_df = new_df.withColumnRenamed(
    "user_id",
    "customer_id"
)

new_df.show()
```

```
+-----+
| city|
+-----+
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
+-----+
only showing top 20 rows
+-----+
| city|
+-----+
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
|Delhi|
+-----+
only showing top 20 rows
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

[6]
✓ 1s

```
from pyspark.sql.functions import count

city_data = [("Delhi",)] * 120 + [("Mumbai",)] * 80

df = spark.createDataFrame(city_data, ["city"])

df.groupBy("city") \
  .agg(count("*").alias("record_count")) \
  .filter("record_count > 100") \
  .show()
```

```
+-----+------------+
| city|record_count|
+-----+------------+
|Delhi|         120|
+-----+------------+
```

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

```
# Sample Data
data = [
    (1, "Active"),
    (2, None),
    (3, "Pending")
]

df = spark.createDataFrame(data, ["id", "status"])

print("Original Data:")
df.show()

# Fill null values
df_filled = df.na.fill("Unknown", subset=["status"])

print("After Filling Null Values:")
df_filled.show()
```

```
Original Data:
+---+-------+
| id| status|
+---+-------+
|  1| Active|
|  2|   NULL|
|  3|Pending|
+---+-------+

After Filling Null Values:
+---+-------+
| id| status|
+---+-------+
|  1| Active|
|  2|Unknown|
|  3|Pending|
+---+-------+
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
4]
4s
```

```python
from pyspark.sql.functions import avg, col

# Sample Data
sales_data = [
    ("West", "Electronics", 500),
    ("West", "Electronics", 700),
    ("West", "Furniture", 300),
    ("East", "Electronics", 900),
    ("West", "Furniture", 500)
]

df_sales = spark.createDataFrame(
    sales_data,
    ["region", "product_category", "sale_amount"]
)

print("Original Data:")
df_sales.show()

# Filter West region and calculate average sales
result = (
    df_sales
    .filter(col("region") == "West")
    .groupBy("product_category")
    .agg(avg("sale_amount").alias("avg_sale_amount"))
)

print("Average Sales in West Region:")
result.show()
```

```
Original Data:
+------+----------------+-----------+
|region|product_category|sale_amount|
+------+----------------+-----------+
|  West|     Electronics|        500|
|  West|     Electronics|        700|
|  West|       Furniture|        300|
|  East|     Electronics|        900|
|  West|       Furniture|        500|
+------+----------------+-----------+

Average Sales in West Region:
+----------------+---------------+
|product_category|avg_sale_amount|
+----------------+---------------+
|     Electronics|          600.0|
|       Furniture|          400.0|
+----------------+---------------+
```

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Q3_DuplicateRemoval").getOrCreate()

# Sample Data
data = [
    (101, "2025-06-01", 500),
    (101, "2025-06-01", 500),  # Duplicate
    (102, "2025-06-02", 750),
    (103, "2025-06-03", 900),
    (103, "2025-06-03", 900)   # Duplicate
]

df = spark.createDataFrame(
    data,
    ["user_id", "transaction_date", "amount"]
)

print("Original Data:")
df.show()

# Remove Duplicates
df_clean = df.dropDuplicates(
    ["user_id", "transaction_date"]
)

print("After Removing Duplicates:")
df_clean.show()
```

```
Original Data:
+-------+----------------+------+
|user_id|transaction_date|amount|
+-------+----------------+------+
|    101|      2025-06-01|   500|
|    101|      2025-06-01|   500|
|    102|      2025-06-02|   750|
|    103|      2025-06-03|   900|
|    103|      2025-06-03|   900|
+-------+----------------+------+

After Removing Duplicates:
+-------+----------------+------+
|user_id|transaction_date|amount|
+-------+----------------+------+
|    101|      2025-06-01|   500|
|    102|      2025-06-02|   750|
|    103|      2025-06-03|   900|
+-------+----------------+------+
```

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

ANS 1 = Traditional MapReduce relies heavily on disk-based processing, which makes it slower for large and complex workloads. It is not efficient for iterative tasks like machine learning because data must be repeatedly read from and written to disk. Spark overcomes these limitations through in-memory processing, faster execution, and easier-to-use APIs.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

ANS 2 - Spark stores intermediate data in memory (RAM) instead of writing it to disk after every operation. This allows machine learning algorithms to reuse data across multiple iterations without repeated disk access, resulting in much faster processing and improved performance.

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

ANS 1 = Traditional MapReduce relies heavily on disk-based processing, which makes it slower for large and complex workloads. It is not efficient for iterative tasks like machine learning because data must be repeatedly read from and written to disk. Spark overcomes these limitations through in-memory processing, faster execution, and easier-to-use APIs.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

ANS 2 - Spark stores intermediate data in memory (RAM) instead of writing it to disk after every operation. This allows machine learning algorithms to reuse data across multiple iterations without repeated disk access, resulting in much faster processing and improved performance.